

算出・判定ロジック仕様（202607）

数値の求め方と自動判定のルールをまとめた仕様。画面から見た機能は [funcspec-202607.md](#)、実装配置は [design-202607.md](#) を参照。

本書は、個別に書かれていた3つの実装仕様書（収束による自動停止 → 3. / 衛星表示のキャリーフォワード → 4. / 受信品質統計 → 5.）を統合したもの。

章	実装
1. 座標とメートルの換算 / 2. 精度と集計 / 3. 収束判定	js/accuracy.js （純粹関数）
3. 収束の適用	js/recorder.js
4. 衛星表示の保持	js/charts.js
5. 受信品質統計	js/stream-stats.js

1. 座標とメートルの換算

局所平面近似で足りる（1 地点のばらつき評価が目的で、扱う距離は数十 m 以下）。

```
latM = 111320 // 南北方向 [m/度]
lonM = 111320 × cos(lat[rad]) // 東西方向 [m/度]
```

中心からのオフセットは [東西](#) $e = (lon - lonMean) \times lonM$ / [南北](#) $n = (lat - latMean) \times latM$ 。

2. 精度の推定と静止点の集計

2-1. 水平精度の推定（1 エポック）

優先順に評価し、最初に成立したものを採用する。

優先	条件	値	表示ソース
1	GST の緯度・経度標準偏差がある	$\sqrt{latStd^2 + lonStd^2}$	GST
2	HDOP がある	$HDOP \times UERE$ （UERE は設定値、既定 5 m）	HDOP×UERE
3	どちらも無い	表示なし（-）	—

GST はレシーバが出力する擬似距離誤差統計に基づく値、HDOP×UERE は概算値である点に注意。

2-2. 記録の集計（全エポック）

対象は「緯度経度がある」エポック。1 件も無ければ集計値は生成しない。

指標	定義
center	緯度・経度の算術平均
median	緯度・経度の中央値（外れ値の影響を見るため併記）
stdEastM / stdNorthM	中心からの東西 / 南北オフセットの標準偏差（不偏、n-1）。平均は 0 として計算
drms	$\sqrt{(\text{stdEastM}^2 + \text{stdNorthM}^2)}$... 水平方向の RMS ばらつき
drms2	$\text{drms} \times 2$ (2DRMS)
cep50 / cep95	中心からの半径誤差を昇順に並べた経験的パーセンタイル（50% / 95%、線形補間）
altMean / altStd	標高(MSL)の平均・標準偏差（値のあるエポックのみ）
count	集計に使ったエポック数
fixCounts	fix quality ごとの件数
avgPdop / avgHdop / avgVdop	各 DOP の平均（値のあるエポックのみ）
avgSats	使用衛星数の平均
avgSnrUsed	使用中の衛星の C/N0 の平均（可視のみの衛星は除く）
offsets	各エポックの {e, n}（散布図用）

- CEP は理論式ではなく**実測分布のパーセンタイル**。したがって点数が少ないと不安定。
- これらは「点のばらつき（精密さ）」であり、真位置からの誤差（正確さ）ではない。

3. 収束による自動停止

3-1. 前提

- MAX-M10S は単周波・自律測位のため、**「収束」＝「これ以上データを足しても推定が動かない（頭打ち）」**であって、**精度の保証ではない**。マルチパスで“安定して偏った”値に張り付くことがあるため、**品質ゲートと上限時間を必ず併用する**。
- 中心と DRMS は累積統計なので時間とともに必ず平坦化する。したがって収束判定は **直近 holdSec 秒の窓**での変動で見る（累積値の単純な差分では早期停止してしまう）。
- 偽収束（マルチパスで偏った値に張り付いたまま安定する）は **minSec** を延ばせば抑えられるが、そのコストは見通しの良い地点にも等しく乗る（開けた場所での所要時間は実質 **minSec** で決まる）。本アプリでは **minSec は 30 秒に据えて偽収束をある程度許容し**、その代わりに 端末内蔵GNSS との同時取得（[funcspeg-202607.md](#) 4-8）と、調査日の対応表CSV による 事後検証（DRMS 比・中心のズレ・平均HDOP）で疑わしい地点を拾う方針とする。

3-2. 品質ゲート

そのエポックを収束判定に使えるかの条件。

```
fixMode === 3 // 3D 測位
&& hdop != null && hdop <= 3
&& satsUsed != null && satsUsed >= 5
```

- 満たさないエポックが来たら**収束用の履歴をリセット**する（「安定してから 10 秒」を数え直す）。
- **記録データは間引かない**。蓄積条件は `lat != null && lon != null && fixQuality > 0` のままで、品質ゲートは停止判定にのみ用いる。

3-3. 判定（毎エポック = 1 Hz）

品質ゲートを通ったエポックについて { `t`: 経過秒, `lat`, `lon`, `drms` }（いずれも**その時点の集計値**）を履歴に積み、次を評価する。

1. `elapsedSec < minSec` または履歴が 2 件未満 → 判定しない (`stable: false`)。
2. `cutoff = elapsedSec - holdSec` 以前で最も新しい履歴を**基準点 ref** とする。存在しなければ窓を満たしていない (`stable: false`、表示は「判定中」)。
3. `centerMoveM` = 基準点と最新の中心間の距離 [m] (1. の換算を使用)。
4. `drmsRangeM` = 窓内 (`t ≥ ref.t`) の DRMS の**最大 - 最小**。
5. 判定：

```
drmsTol = max(drmsTolAbsM, drmsTolPct × 現在のDRMS)
stable = (centerMoveM <= centerTolM) && (drmsRangeM <= drmsTol)
```

履歴は基準点を確保できる範囲で間引く (`holdSec + 5` 秒より古いものを捨てる)。

3-4. パラメータ

名前	既定値	種別	説明
<code>autoStop</code>	<code>true</code>	設定 (<code>js/constants.js</code>)	収束による自動停止 ON/OFF
<code>minSec</code>	30 秒	設定 (<code>js/constants.js</code>)	これ未満では絶対に停止しない
<code>maxSec</code>	180 秒	設定 (<code>js/constants.js</code>)	上限＝タイムアウト。0 で無制限
<code>maxEpochs</code>	300	設定 (<code>js/constants.js</code>)	上限エポック数。0 で無制限
<code>holdSec</code>	10 秒	定数	収束が維持されるべき時間（判定窓）
<code>centerTolM</code>	0.3 m	定数	窓内の中心の許容移動量
<code>drmsTolAbsM</code>	0.3 m	定数	DRMS の許容変動（絶対）
<code>drmsTolPct</code>	0.05 (5%)	定数	DRMS の許容変動（相対）

定数は画面の煩雑化を避けるためモジュール定数 (`js/recorder.js` の `CONVERGENCE`) とし、設定画面には出さない。一方「設定」種別の既定値は `js/constants.js` の `DEFAULT_SETTINGS` にまとめてあり、設定タブから起動中のみ変更できる（永続化しない → `funcspec-202607.md` 7.)。

3-5. 停止理由

停止時に `stopReason` を記録し、結果テキストの先頭に表示する。

値	契機	表示
<code>converged</code>	3-3. の <code>stable</code> が成立	自動停止: 収束（中心・DRMS横ばい）
<code>timeout</code>	<code>maxSec > 0 && elapsedSec >= maxSec</code>	自動停止: 上限時間到達（未収束）
<code>maxEpochs</code>	<code>maxEpochs > 0 && 点数 >= maxEpochs</code>	自動停止: 上限エポック到達
<code>manual</code>	停止ボタン	手動停止

4. 衛星表示のキャリーフォワード

4-1. 課題

スカイプロットと SNR チャートは毎回クリアしてから描くため、GSV が届かない秒には 表示が空になり点滅して見える（GSV は帯域や欠落の影響で每秒そろわないことがある）。

4-2. 判定（純粋関数 `holdDecision`）

```
holdDecision(hasSats, ageMs, holdMs):
  hasSats          → 'draw'      // 描画可能な衛星データがある：通常描画
  ageMs <= holdMs   → 'hold'      // 失効時間内：前フレームを保持（クリアも描画もしない）
  それ以外          → 'clear'     // 失効：枠（グリッド/目盛り）だけ描いてクリア
```

記号	意味
<code>hasSats</code>	スカイプロットは仰角・方位を持つ衛星、SNR は C/N0 を持つ衛星が 1 つ以上あるか
<code>ageMs</code>	最後に有効データを描画してからの経過（未受信は ∞ → 初期状態は <code>clear</code> ）
<code>holdMs</code>	保持上限 <code>HOLD_MS = 8000</code> （両ビュー共通の定数）

- 経過時間の基準はエポックの受信時刻（`recvAt`）。読込データを表示する場合は 表示時刻を `recvAt` として補うため、ライブと同じ経路で判定される。
- 保持中のリサイズ・タブ再表示では、**最後の有効エポック**で再描画する（`redraw()`）。
- 系統別チップも、内訳が空のときは前の表示を保持する。

5. 受信品質統計

5-1. 考え方

`M10S --UART--> Pico --BLE(NUS)--> アプリ` の各段で「入った数」と「出た数」を数え、区間差分を突き合わせれば、どの段で・どれだけ取りこぼしたかが分かる。

M10S —UART→ Pico —BLE notify→ LineBuffer → parser → EpochAssembler
 欠落モード: ①UART溢れ/化け ②BLE送信破棄 ③通知消失 ④CS NG ⑤未対応文 ⑥工
 ポック/GSV欠け
 検出手段: Pico側 ng/drop Pico側 txng Δtxok-Δ受信行数 csNg計数 unknown計数
 時刻ギャップ/msgNum

Pico 側のカウンタ \$PPICO の仕様は [hardware-202607.md](#) 4. を参照。

5-2. アプリ側のカウンタ

`reset()`（接続開始・モック開始・手動リセット）起点の累計。1 秒ごとに接続タブへ描画する。

カウンタ	供給元	意味
<code>lines</code>	受信フレーム	LineBuffer が復元した全行（\$PPICO 含む）
<code>csNg</code>	パーサ	チェックサム不一致（ 計数して破棄 ）
<code>parsedOk</code>	パーサ	内容まで解釈した行（GGA/RMC/GSA/GSV/VTG/GST）
<code>unknown</code>	パーサ	チェックサム正常だが未対応の種別（ 計数のみで無視 ）
<code>discardedChars</code>	LineBuffer	行にならないまま捨てた文字数（4096 字超で破棄）
<code>epochs / epochGaps</code>	エポック確定	確定エポック数 / 時刻ギャップ（1.5 秒超）から推定した欠落エポック数
<code>gsvMissing</code>	エポック確定	GSV グループ（talker × signalId）で届かなかった msgNum の数
<code>pico.* / picoSeqGaps</code>	\$PPICO	Pico 側カウンタの最新値 / <code>seq</code> 抜けの数
<code>bleLossEst</code>	突合	BLE 通知欠落の推定（下記）

5-3. BLE 欠落の推定

最初の \$PPICO 受信時点の `txok` とアプリの受信行数を基準点として対応付け、以降は

$$\text{bleLossEst} = (\text{txok} - \text{txok基準}) - (\text{受信行数} - \text{受信行数基準}) \quad \text{※負は 0 に丸める}$$

- 化けて届いた行も「受信した行」として数える（`csNg` 行き）ため、これは**通知ごと消えた分**の推定。
- BLE の in-flight データの分だけ基準点に数行のずれが乗り得るので、±数行は誤差とみなす。
- `txok` または `seq` が後退したら Pico 再起動とみなし、基準点を取り直す（推定値は 0 に戻る）。
- \$PPICO を 1 度も受けていなければ `null`（表示は -）。

5-4. 取りこぼし時の対応

事象	対応
チェックサム NG 行	<code>csNg</code> に計数して破棄。接続・処理は継続
未対応センテンス	<code>unknown</code> に計数のみ
行にならないゴミ	LineBuffer が 4096 字超で破棄し、文字数を <code>discardedChars</code> に計上
<code>\$PPICO</code>	統計にのみ反映し、エポック組み立てには回さない
描画が追いつかない	<code>requestAnimationFrame</code> で最新エポックのみ描画（データ処理自体は間引かない）

BLE notify はブラウザのイベントキューに積まれるため、アプリの処理が一時的に遅れても データは失われない（失われるのは BLE リンク層で、それは `bleLossEst` に現れる）。

5-5. 記録への保存（区間差分）

記録 1 回分の区間差分（停止時の値 - 開始時の値）を `summary.rxStats` に保存し、結果テキストの末尾に表示する。区間開始時に `$PPICO` を未受信だった場合、Pico カウンタの差分は帰属できないため `pico: null`（表示は「Pico統計（\$PPICO）なし」とする）。

5-6. 注意

- 接続直後の初回エポックは途中受信のため GSV 部分欠落に 1 回分計上され得る（誤差として無視してよい）。
- モック配信も `$PPICO` を模擬する（欠落ゼロの整合値 → BLE 欠落推定は 0 になるはず）。
- Pico のカウンタは起動からの累計で、切断・再接続ではリセットされない（アプリ側が差分を取る）。
- **すべて 0（BLE 欠落 0・Pico NG 0）なら「M10S → Pico → アプリまで取りこぼしなし」と確認できる。**

6. テストで担保している項目

`node tools/check-syntax.mjs`（純粋関数の単体テスト）で次を検証している。

対象	検証内容
チェックサム	正しい文の受理・不正な文の棄却
パース	GGA の座標変換・フィールド、GST、VTG、 <code>\$PPICO</code> 、GSV の <code>signalId</code>
行復元	チャンク断片の結合、溢れ破棄の通知
精度推定	GST 優先、HDOP×UERE フォールバック
集計	点数・平均緯度・DRMS/CEP の大小関係・散布図オフセット・使用衛星の平均 C/N0・m/度換算
収束判定	横ばい列で成立／ドリフト列・DRMS 変動列・時間不足・窓未充足で不成立
表示保持	<code>holdDecision</code> の 4 パターン

対象	検証内容
受信品質	行分類・BLE 欠落推定・\$PPICO 欠落検出・Pico 再起動・エポックギャップ・区間差分
記録フロー	stop で集計のみ（未保存）・衛星リストの保持・save で保存・summary の集計値
既定地点名	同日連番の採番規則